Programming Paradigm Analysis

Project: Java OOP Passenger Van Rental System

Comparison of programming paradigms

Dimension	Procedural	Functional	Object-oriented
Main unit	Procedures that work on data	Functions that take values and return new values	Objects that hold data and behavior together
State	Often changed by procedures	Prefers values that do not change	Kept inside each object and changed through its methods
Usual flow	Steps, choices and loops	Putting functions together	Objects calling each other's methods
Reuse	Shared functions and modules	Reusable pure functions	Interfaces, composition and inheritance that makes sense
Van rental example	rentVan(vans, customers, rentals, data)	validate(request), then return a new list of rentals	RentalSystem makes a Rental linked to a Customer and a Van, and the van works out its own price
Strength	Simple for small tasks	Easy to test, since the same input gives the same output	Matches real things and lets parts be replaced
Tradeoff	Shared data can make parts depend on each other too much	Input, output and changing data need to be kept apart	Can be too much for a small program if classes have no clear job

Selection

We chose OOP as the main style. The system has data that must be saved, details shared by different kinds of people, van types with different prices, links between rentals, customers and vans, and status changes such as rented, returned and in maintenance. Procedural style is still used in the menu loop and the input prompts in Main. Functional style would also work well for small jobs like checking input and counting late days. Choosing OOP does not stop us from using these ideas.

Applying each paradigm to renting a van

Procedural approach

A rental procedure would get the van, customer and rental lists. It would find the IDs, check the van flags, compute the fee with a check on the van type, and add a record. This works

for a small script. But every procedure must agree on how the data looks and what each flag means. When we add returns, maintenance and overdue checks, the same checks get copied in many places and may become different from each other.

Functional approach

A check function could return an error or a valid request. A pure rental function could take the current rentals and return a new list that includes the new rental. The old list stays the same. A late fee function that takes the end date, the return date and the rate would be easy to test alone. This is easier to predict. But reading files, typing input and changing van status still need to be done at the edge of the program.

Object oriented approach in the supplied Java code

Main reads the staff ID, the customer ID and the plate number. RentalSystem.rentVan finds the Customer and checks that ID is on file. Then it calls checkVanForRent and makes a Rental that holds the Customer, the Van and the Employee. The Rental constructor asks the Van for its fee with calculateFee. So StandardVan and LuxuryVan each use their own price. After that, RentalSystem marks the van as not available, saves the rental in the list and sends the receipt through a Notification. Returns and maintenance change the same objects with methods. This keeps the link between van, customer and rental clear.

Design choice	Reasonable purpose	Current limitation
Abstract Person	Keep the name, contact number and email in one place for Customer and Employee.	Person has no setter for the name. Its setters do not check anything. The checking is done in Main.
Abstract Van with StandardVan and LuxuryVan	Share the plate, model, seats and status code. Each subclass gives its own rate and fee.	Rates and the cleaning fee are fixed numbers. Status uses two flags (available and under maintenance) that RentalSystem must keep correct.
Notification interface	Let different senders use the same send(recipient, message) method.	Email and SMS only print and add to a log file. Both classes have the same logging code and file name.
RentalSystem service class	Keep the record work and the rules away from the menus.	Its getters return the real lists. Rental and maintenance IDs come from the list size, so they could repeat if records were deleted.
FileManager	Keep all file formats in one class and rebuild the links between rentals, customers, vans and staff.	Uses fixed file names and comma-separated lines. A rental or maintenance record with a missing van or customer is skipped. One bad line can stop the rest of that file from loading.

RentalException	Show a clear message when a rule is broken. Main catches it in one place.	It is one general exception with no codes to tell the cases apart.

Justification for Using OOP in the Passenger Van Rental System

Concept	Concrete source evidence	Explanation
Class vs object	The Customer class; new Customer(...) in RentalSystem.addCustomer()	A class describes fields and behavior. Each customer made from it has its own values.
Encapsulation	Private fields in Person, Van and Rental with public getters	Other code uses methods and does not touch the fields. Exception: getVans(), getCustomers() and getRentals() return lists that can be changed.
Inheritance	Customer and Employee extend Person; StandardVan and LuxuryVan extend Van; RentalException extends Exception	Each pair shares some data or behavior and adds its own. Customer adds ID and address. Each van type adds its own rate and fee rule.
Abstraction	Person.getRole(); Van.getDailyRate(), calculateFee() and getVanType(); Notification.send(...)	The abstract class or interface sets what must be done. The other classes decide how.
Polymorphism	Rental calls van.calculateFee(days) on a Van reference; RentalSystem puts EmailNotification and SmsNotification in Notification fields	One call runs the code of the real object. Luxury adds the cleaning fee. Email and SMS show their messages differently.
Association	Rental holds Customer, Van and Employee; MaintenanceRecord holds Van and Employee	A record links things that already exist. It does not control how long they exist. A customer with rentals cannot be deleted.
Composition and aggregation	RentalSystem creates its own FileManager and two Notification objects; its ArrayLists hold Van, Customer, Rental, Employee and MaintenanceRecord objects	RentalSystem owns its helper objects (composition). The lists hold things that rentals and records also point to (aggregation).
Exception handling	RentalSystem throws RentalException; Main.main catches it and prints the message	When a rule is broken, the action stops and the menu comes back. The program does not crash.

Conclusion

Procedural style is simple, but it gets hard to manage when the program grows. Functional style is good for working on data, but it does not fit well with things that keep changing, like the status of a van. OOP fits our van rental system best. Customers, vans, rentals, staff and maintenance jobs are real things, so each one became a class. Inheritance saved us from repeating the same fields. Polymorphism let each van type work out its own fee and let email and SMS send messages in their own way. Encapsulation kept the data inside the objects. OOP also has some weak points in our code. The daily rates are fixed numbers, and some getters return the real lists. Even so, OOP is the right choice for this project. We can still use functional ideas, like streams, inside the OOP design when we need to filter or add up rentals.